

Production-Grade Real-Time Data Streaming Infrastructure Design

Candidate: Amirmohammad Shakeri

Email: amirmuhammadshk@gmail.com

1. Executive Summary

This document proposes a production-grade real-time data streaming infrastructure for market data, news, sentiment, and computed ML features.

The system is designed for an algorithmic trading and ML-driven environment where infrastructure reliability, data correctness, low-latency delivery, historical replay, and observability are critical. Downstream inference and decision-making systems depend on the data produced by this platform, so the architecture prioritises correctness, durability, operational visibility, and controlled failure behaviour.

The proposed solution uses a streaming-first architecture on AWS. Amazon MSK, a managed Kafka-compatible service, acts as the central event backbone. Dedicated ingestion services validate and normalise market and news data before publishing to Kafka topics. Apache Flink stream-processing jobs compute aggregated candles and real-time features. Low-latency serving systems expose recent features and indicators to online inference services, while S3-backed Apache Iceberg tables provide historical data for training, backtesting, and offline analytics.

2. Requirements

The platform must support the following input data:

- 1-minute OHLCV candles for 3,000 assets, 24 hours per day.
- Derived candles for 15-minute, 30-minute, 1-hour, and 1-day timeframes.
- News and sentiment events, averaging 20-30 events per asset per day.
- 200 computed features per asset derived from market data, news, and sentiment.

The platform must reliably serve the following consumers:

- Online serving for low-latency inference and decision-making.
- Real-time feature computation for continuously updated features and indicators.

- Offline and training pipelines requiring historical and recent data.
- Monitoring and observability systems for infrastructure and data health.

3. Assumptions

The following assumptions are made for this design:

Area	Assumption
Cloud provider	AWS is used as the production cloud platform.
Streaming model	Kafka-compatible event streaming is preferred for replay, ordering, and fan-out.
Ordering	Strict ordering is required per asset, not globally across all assets.
Partition key	Market data and features are partitioned by <code>asset_id</code> .
Latency	Online feature updates should normally be available within seconds of source arrival.
Schemas	Events are encoded using Avro or Protobuf with schema compatibility checks.
Data retention	Kafka stores short-to-medium-term replay data; S3 stores long-term historical data.
Backfill	Historical replay and backfills must be supported without disrupting live processing.
Security	All services use private networking, encryption in transit, encryption at rest, and least-privilege IAM.
Deployment	Infrastructure is managed through Terraform and deployed through CI/CD.

4. Data Volume Estimate

4.1 Market Data

For 3,000 assets with one 1-minute candle per asset:

```
3,000 records per minute
50 records per second
4.32 million records per day
```

The raw market data volume is moderate, but the operational requirements are strict because the data feeds trading and ML decision systems.

4.2 News and Sentiment

Assuming 20-30 events per asset per day:

$$3,000 \text{ assets} * 20-30 \text{ events/day} = 60,000-90,000 \text{ news events/day}$$

Coverage is uneven across assets, so the design must handle bursts for highly covered assets while avoiding assumptions of uniform traffic distribution.

4.3 Computed Features

For 200 features per asset:

$$3,000 \text{ assets} * 200 \text{ features} = 600,000 \text{ latest feature values}$$

If features are refreshed every minute:

$$\begin{aligned} &600,000 \text{ feature values per minute} \\ &10,000 \text{ feature values per second} \end{aligned}$$

This is manageable with horizontally scalable stream processing and a low-latency online feature store.

5. High-Level Architecture

The architecture is built around a durable streaming backbone.

External Providers

-> Ingestion Services

-> Amazon MSK / Kafka

-> Stream Processing

-> Online Serving, Historical Storage, Training Pipelines, Monitoring

The main components are:

Component	Technology	Purpose
Market data ingestion	Containerised service on ECS or EKS	Validate, normalise, deduplicate, and publish OHLCV events.
News ingestion	Containerised service on ECS or EKS	Validate, enrich, deduplicate, and publish news/sentiment events.

Component	Technology	Purpose
Streaming backbone	Amazon MSK / Kafka	Durable event log, replay, ordering, and multi-consumer fan-out.
Schema registry	AWS Glue Schema Registry or Confluent Schema Registry	Schema versioning and compatibility control.
Stream processing	Apache Flink	Candle aggregation, feature computation, windowed joins, and late-data handling.
Online feature store	DynamoDB or ElastiCache Redis	Low-latency access to latest features for inference.
Time-series serving	Amazon Timestream or optimised DynamoDB model	Recent candles and indicators for fast online queries.
Data lake	S3 with Apache Iceberg	Raw, cleaned, and curated historical data for training and backtesting.
News search	OpenSearch, optional	Search and audit over news headline/body fields.
Observability	CloudWatch, Prometheus, Grafana	Infrastructure metrics, application metrics, stream lag, and data freshness.
Alerting	CloudWatch alarms, SNS, PagerDuty integration	Production incident notification.

6. Data Flow

6.1 Market Data Flow

Market data providers send 1-minute OHLCV candle events to the market ingestion service.

The ingestion service performs:

- Schema validation.
- Type and range validation.
- Timestamp normalisation.
- Asset identifier validation.
- Deduplication using source, asset, and timestamp.
- Idempotent publishing to Kafka.

Valid events are published to a topic such as:

```
market.ohlcv.1m.v1
```

Invalid events are routed to a dead-letter topic such as:

```
dlq.market.ohlcv.1m.v1
```

6.2 News and Sentiment Flow

News and sentiment events are processed by a separate ingestion service because they have different data shape, traffic pattern, and validation requirements.

Valid events are published to:

```
news.sentiment.raw.v1
```

The system stores news text in the historical data lake and may index headline/body fields into OpenSearch for audit and search use cases.

6.3 Stream Processing Flow

Flink jobs consume Kafka topics and produce:

- 15-minute, 30-minute, 1-hour, and 1-day candles.
- Rolling technical indicators.
- News-derived and sentiment-aware features.
- Latest feature vectors per asset.

Processed outputs are written to:

- Kafka topics for downstream real-time consumers.
- Online serving stores for inference.
- S3 Iceberg tables for training and backtesting.

7. Kafka Topic Design

A proposed topic structure is:

Topic	Key	Purpose
<code>market.ohlcv.1m.v1</code>	<code>asset_id</code>	Raw validated 1-minute market candles.
<code>news.sentiment.raw.v1</code>	<code>asset_id</code> or news event id	Validated news and sentiment events.
<code>market.candles.15m.v1</code>	<code>asset_id</code>	Derived 15-minute candles.

Topic	Key	Purpose
<code>market.candles.30m.v1</code>	<code>asset_id</code>	Derived 30-minute candles.
<code>market.candles.1h.v1</code>	<code>asset_id</code>	Derived hourly candles.
<code>market.candles.1d.v1</code>	<code>asset_id</code>	Derived daily candles.
<code>features.realtime.v1</code>	<code>asset_id</code>	Latest computed features for online consumers.
<code>dlq.market.v1</code>	original key	Invalid or poison market events.
<code>dlq.news.v1</code>	original key	Invalid or poison news events.

Partitioning by `asset_id` preserves per-asset ordering and allows horizontal scaling across consumers.

8. Stream Processing Design

Apache Flink is used for stateful stream processing because it supports event-time processing, watermarks, windowing, state management, and exactly-once style processing patterns when configured with compatible sinks.

The platform uses separate jobs for:

1. Candle aggregation.
2. Feature computation.
3. Data-quality monitoring.
4. Data lake sinking.

This separation reduces blast radius. A failure in an optional sink should not stop the core market-data stream.

8.1 Candle Aggregation

The candle aggregation job consumes `market.ohlcv.1m.v1` and computes:

- 15-minute candles.
- 30-minute candles.
- 1-hour candles.
- 1-day candles.

The job uses event time rather than processing time. This allows late-arriving events to be handled correctly within a configured lateness window.

8.2 Feature Computation

The feature job consumes market data, aggregated candles, news, and sentiment events.

It computes 200 features per asset and writes the latest feature vector to:

```
features.realtime.v1
```

It also writes serving-ready feature values to the online store.

8.3 Late and Missing Data

Late data is handled using watermarks and bounded lateness windows. If data arrives after the correction window, the system emits a correction event rather than silently overwriting historical results.

Missing data is detected through data-quality jobs that track expected candle arrivals per asset and timeframe.

9. Storage Design

9.1 Raw Zone

The raw S3 zone stores immutable event archives exactly as received after ingestion validation.

Example layout:

```
s3://platform-data/raw/source=market_data/date=YYYY-MM-DD/hour=HH/  
s3://platform-data/raw/source=news/date=YYYY-MM-DD/hour=HH/
```

This enables audit, replay, and reprocessing.

9.2 Curated Zone

The curated zone stores cleaned and queryable Iceberg tables:

```
market_candles_1m
market_candles_15m
market_candles_30m
market_candles_1h
market_candles_1d
news_sentiment
features_historical
```

Iceberg is used because it supports reliable table metadata, partition evolution, schema evolution, and reproducible historical reads.

9.3 Online Serving Store

The online serving store contains the latest feature vector per asset.

Example key model:

```
PK: asset_id
SK: feature_set_version
attributes: feature values, event timestamp, computed timestamp, source c
```

For very low-latency inference, Redis can be used. For simpler durability and operational management, DynamoDB can be used. In production, the final choice depends on latency targets, access patterns, durability needs, and operational preference.

10. Reliability and Failure Behaviour

The system is designed to fail safely.

10.1 Ingestion Failures

If a provider sends invalid data, the event is rejected to a dead-letter topic with failure metadata.

The ingestion service should not crash on bad input. It should isolate bad records and continue processing valid events.

10.2 Kafka or MSK Issues

MSK is deployed across multiple availability zones. Producers use retries, acknowledgements, and idempotent publishing where appropriate.

Consumers track offsets and can resume from committed positions after restart.

10.3 Stream Processing Failures

Flink jobs use checkpoints. On restart, jobs resume from the latest checkpoint.

Sinks are designed to be idempotent where possible. For example, writes include deterministic keys based on asset, timestamp, timeframe, and feature version.

10.4 Downstream Store Failures

If the online store is temporarily unavailable, feature events remain in Kafka and can be replayed by the serving sink once the store recovers.

The system should prefer delayed delivery over silent data loss.

11. Data Correctness

Data correctness is enforced through multiple layers:

- Schema validation before publishing.
- Deduplication using deterministic event identifiers.
- Per-asset ordering through Kafka partitioning.
- Event-time processing for windowed computations.
- Data-quality checks for missing, late, duplicate, or stale data.
- Idempotent writes to serving and historical stores.
- Audit metadata including source timestamp, ingestion timestamp, processing timestamp, schema version, and Kafka offset.

For trading and ML systems, data correctness is as important as service availability. Incorrect data can be more dangerous than unavailable data because it may cause downstream systems to make bad decisions.

12. Observability

The platform exposes both infrastructure and data-quality observability.

12.1 Infrastructure Metrics

Key metrics include:

- Kafka broker CPU, memory, disk usage, and network throughput.
- Kafka consumer lag by topic and consumer group.
- Producer error rate.
- Flink checkpoint duration and failure count.
- Flink backpressure.
- Online store latency and throttling.
- S3 sink failure rate.
- Error and retry rates by service.

12.2 Data Health Metrics

Data-specific metrics include:

- Last successful candle timestamp per asset.
- Missing candle count per asset and timeframe.
- Duplicate event count.
- Late event count.
- Feature freshness by asset.
- News ingestion delay by provider.
- Number of events in dead-letter topics.
- Difference between source timestamp and processing timestamp.

12.3 Alerting

Example alerts:

Alert	Condition
Market data stale	No new candle for critical assets within expected interval.
Kafka lag high	Consumer lag exceeds threshold for production topics.
Flink checkpoint failures	Consecutive checkpoint failures above threshold.
DLQ spike	Invalid event count rises suddenly.
Feature freshness breach	Latest feature timestamp is older than allowed SLA.
Online store latency high	P95 or P99 latency exceeds threshold.

13. Security

Security is applied throughout the platform:

- Private subnets for MSK, Flink, ingestion services, and serving stores.

- TLS encryption in transit.
- Encryption at rest using KMS-managed keys.
- Least-privilege IAM roles.
- Security groups restricting traffic by service role.
- Secrets stored in AWS Secrets Manager or SSM Parameter Store.
- No public access to Kafka brokers or internal storage.
- CI/CD uses short-lived credentials where possible.
- Terraform state stored in a secured remote backend in production.

14. Scalability

The platform scales horizontally.

Kafka topics are partitioned by asset. Flink parallelism can be increased to process more partitions. Ingestion services can scale by provider or source. Online serving capacity can scale independently from offline training workloads.

The initial data volume is not extremely large, but the design allows growth in:

- Number of assets.
- Number of features.
- Number of news providers.
- Higher-frequency market data.
- Additional consumers.
- Additional asset classes.

15. Tradeoffs

15.1 Kafka/MSK vs Kinesis

Kafka/MSK is selected because it provides a strong event-streaming model with topic-based fan-out, replay, consumer groups, and broad ecosystem support. Kinesis would also be a valid AWS-native option, but Kafka is a better fit for a multi-consumer, replay-heavy trading and ML platform.

15.2 DynamoDB vs Redis for Online Serving

DynamoDB provides managed durability, predictable scaling, and simple operational behaviour. Redis provides lower latency and richer in-memory access patterns, but requires more careful memory and failover management. For the final production system, the choice depends on the strictness of inference latency requirements.

15.3 Single Large Flink Job vs Multiple Smaller Jobs

A single job can reduce duplicated reads, but multiple jobs provide better isolation, easier deployment, and lower blast radius. This design favours multiple jobs for production operability.

15.4 Raw and Curated Storage

Raw storage is kept immutable for audit and replay. Curated Iceberg tables are optimised for query, training, and backtesting. This increases storage duplication but improves reliability, reproducibility, and operational clarity.

16. Terraform Component Choice

The Terraform code in this submission focuses on the Amazon MSK streaming layer.

This component was chosen because Kafka/MSK is the central backbone of the architecture. It directly supports:

- Reliable ingestion.
- Durable event storage.
- Multiple independent consumers.
- Replay and backfill.
- Per-asset ordering.
- Real-time feature computation.
- Decoupling between producers and consumers.

A production-grade streaming layer is critical for the reliability of the entire platform.

17. Production Deployment Behaviour

In production, the platform would be deployed through CI/CD.

A normal deployment flow would include:

1. Terraform formatting and validation.
2. Static security checks.
3. Plan generation.
4. Manual approval for production changes.
5. Terraform apply.
6. Post-deployment smoke tests.
7. Monitoring dashboards and alert validation.

Application deployments would use rolling or blue/green deployment patterns. Stream processing jobs would be deployed carefully to avoid offset loss, schema incompatibility, or accidental state reset.

18. Backfill and Replay Strategy

The system supports replay in two ways:

1. Kafka replay for recent events within retention.
2. S3 raw-zone replay for long-term historical reprocessing.

Backfill jobs should publish replayed events to separate backfill topics or include explicit metadata identifying the run. This prevents historical correction jobs from accidentally mixing with live traffic without control.

19. Disaster Recovery

The system is designed to be recreated from infrastructure-as-code.

Key DR considerations:

- Terraform defines core infrastructure.
- Kafka topics and configuration are version controlled.
- S3 data is durable and versioned where required.
- Critical metadata is backed up.
- Deployment pipelines can rebuild environments.
- Runbooks define recovery steps for broker failure, data lag, bad deployment, and corrupted downstream data.

20. Conclusion

The proposed architecture provides a reliable, scalable, and observable real-time streaming platform for market data, news, sentiment, and ML feature delivery.

It separates ingestion, streaming, processing, serving, historical storage, and monitoring concerns. This allows the system to support low-latency online inference, real-time feature computation, historical model training, and production observability without tightly coupling consumers to producers.

The design prioritises correctness, replayability, operational visibility, and controlled failure behaviour, which are essential properties for infrastructure supporting algorithmic trading and ML-driven systems.